1. Palindrome

A palindrome is a string or sequence that reads the same forward and backward (ignoring spaces, punctuation, and case). In an interview, you'd typically be asked to check if a given string is a palindrome.

Detailed Explanation

- **Approach**: Compare characters from the start and end of the string, moving toward the center. If all pairs match, it's a palindrome. Handle edge cases like empty strings (true) or single characters (true).
- **Time Complexity**: O(n), where n is the string length.
- **Space Complexity**: O(1) for in-place comparison.

Java Code Example

Java

```
public class PalindromeChecker {
    public static boolean isPalindrome(String str) {
        if (str == null || str.length() == 0) {
            return true; // Edge case
        }
        str = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", ""); // Clean
non-alphanumeric chars
        int left = 0;
        int right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        String input = "A man a plan a canal Panama";
        System.out.println(isPalindrome(input)); // Output: true
    }
}
```

- **How it works**: The while loop checks pairs from both ends. Cleaning the string handles real-world inputs.

2. Find Unique Characters in a Char Array Using For Loop

Given a char array with duplicates (e.g., char[] arr = {'a', 'b', 'a', 'c', 'b'};), find and print only unique characters. Use only a for loop (no advanced data structures like HashSet for simplicity, as per "for loop" constraint).

Detailed Explanation

- **Approach**: Use nested for loops to compare each character with all others. If no match is found, it's unique. Track visited indices to avoid re-checking.
- **Time Complexity**: $O(n^2)$, where n is array length (acceptable for small arrays).
- **Space Complexity**: O(1).

Java Code Example

Java

```java
public class UniqueChars {
    public static void findUniqueChars(char[] arr) {
        int n = arr.length;
        boolean[] visited = new boolean[n]; // Track if char is processed

        for (int i = 0; i < n; i++) {
            if (visited[i]) continue; // Skip if already processed

            boolean isUnique = true;
            for (int j = i + 1; j < n; j++) {
                if (arr[i] == arr[j]) {
                    isUnique = false;
                    visited[j] = true; // Mark duplicate as visited
                    break;
                }
            }
            if (isUnique) {
                System.out.print(arr[i] + " ");
            }
            visited[i] = true;
        }
    }

    public static void main(String[] args) {
        char[] arr = {'a', 'b', 'a', 'c', 'b'};
```

```
        findUniqueChars(arr); // Output: c
    }
}
```

- **How it works**: Outer loop picks a char; inner loop checks for duplicates ahead. If none, print it. visited prevents redundant checks.

3. Swapping Program

Swap two variables without using a temporary variable (common twist). Basic version uses temp; advanced uses arithmetic or XOR.

Detailed Explanation

- **Basic Approach (with temp)**: Use a third variable.
- **Without Temp (Arithmetic)**: For integers, a = a + b; b = a - b; a = a - b;.
- **Without Temp (XOR)**: a = a ^ b; b = a ^ b; a = a ^ b; (bitwise, works for ints).
- **Limitations**: Arithmetic can cause overflow; XOR doesn't work for same values.

Java Code Example (Multiple Methods)

Java

```
public class Swapping {
    // With temp
    public static void swapWithTemp(int a, int b) {
        int temp = a;
        a = b;
        b = temp;
        System.out.println("With temp: a=" + a + ", b=" + b);
    }

    // Without temp (arithmetic)
    public static void swapArithmetic(int a, int b) {
        a = a + b;
        b = a - b;
        a = a - b;
        System.out.println("Arithmetic: a=" + a + ", b=" + b);
    }

    // Without temp (XOR)
    public static void swapXOR(int a, int b) {
        a = a ^ b;
        b = a ^ b;
        a = a ^ b;
```

```
        System.out.println("XOR: a=" + a + ", b=" + b);
    }

    public static void main(String[] args) {
        int x = 10, y = 20;
        swapWithTemp(x, y);
        x = 10; y = 20; // Reset
        swapArithmetic(x, y);
        x = 10; y = 20; // Reset
        swapXOR(x, y);
    }
}
```

- **Output**:

  text

```
With temp: a=20, b=10
Arithmetic: a=20, b=10
```

- `XOR: a=20, b=10`
- **How it works**: Each method achieves swap; choose based on constraints (e.g., no overflow for arithmetic).

4. SQL Problems on DELETE, GROUP BY, HAVING (Basic Examples)

Assuming basic problems like deleting duplicates, grouping sales by category with conditions. I'll provide 4-5 example queries with explanations. Use a sample table: employees (id, name, dept, salary).

Detailed Explanation

- **DELETE**: Removes rows matching conditions. Use WHERE for specifics.
- **GROUP BY**: Aggregates data (e.g., COUNT, SUM) by column.
- **HAVING**: Filters groups (like WHERE but for aggregates).

Example Queries

1. **Delete employees with salary < 50000**:

   SQL

2.

   ```
   DELETE FROM employees WHERE salary < 50000;
   ```

- **Explanation**: Basic DELETE with WHERE.

3. **Group by department, count employees, having > 5**:

   SQL

```
SELECT dept, COUNT(*) AS emp_count
FROM employees
GROUP BY dept
```

4. `HAVING COUNT(*) > 5;`
   - **Explanation**: Groups by dept; HAVING filters groups with >5 employees.
5. **Delete duplicates (keep one per name)**:

   SQL

```
DELETE e1 FROM employees e1
INNER JOIN employees e2
```

6. `WHERE e1.id > e2.id AND e1.name = e2.name;`
   - **Explanation**: Self-join to identify duplicates; delete higher ID.
7. **Group salaries by dept, sum > 100000**:

   SQL

```
SELECT dept, SUM(salary) AS total_salary
FROM employees
GROUP BY dept
```

8. `HAVING SUM(salary) > 100000;`
   - **Explanation**: Aggregates sum per group; HAVING filters high totals.
9. **Delete from multiple conditions (e.g., low salary in specific dept)**:

   SQL

```
DELETE FROM employees
```

10. `WHERE salary < 40000 AND dept = 'IT';`
    - **Explanation**: Combines WHERE clauses with AND.

5. What is Stored Procedure? What is "IN", "OUT"?

Detailed Explanation

- **Stored Procedure**: A pre-compiled SQL code block stored in the database. It can accept parameters, perform operations (queries, updates), and return results. Benefits: Reusability, performance (compiled once), security (hides logic).
    - Syntax: CREATE PROCEDURE proc_name(parameters) AS BEGIN ... END;
    - Example Use: CALL proc_name(args);
- **IN Parameter**: Input-only; passes data to the procedure (default mode). Value can't be changed inside.
- **OUT Parameter**: Output-only; returns data from procedure to caller. Procedure sets its value.
- **INOUT**: Both input and output; can read/modify.

SQL Example (MySQL)

SQL

```
DELIMITER //
CREATE PROCEDURE GetEmployeeSalary(IN emp_id INT, OUT total_salary
DECIMAL(10,2))
BEGIN
    SELECT salary INTO total_salary
    FROM employees
    WHERE id = emp_id;
END //
DELIMITER ;

-- Call
CALL GetEmployeeSalary(101, @sal);
SELECT @sal; -- Outputs the salary
```

- **How it works**: IN emp_id fetches data; OUT total_salary returns it.

6. What is View? Memory Cache

Detailed Explanation

- **View**: A virtual table based on a SQL query. It doesn't store data (unless materialized); simplifies complex queries, enhances security (hide columns). Syntax: CREATE VIEW view_name AS SELECT ...;
    - Pros: Reusability, abstraction.
    - Cons: Can be slower for large data.
- **Memory Cache**: Temporary storage in RAM for frequently accessed data to speed up retrieval (e.g., Redis, Ehcache). Reduces database hits.
    - **How it Works**: Key-value store; cache hit = fast read; miss = fetch from source and store.

    - **Types**: In-memory (volatile), distributed.

SQL View Example

SQL

```
CREATE VIEW HighSalaryEmps AS
SELECT name, salary
FROM employees
WHERE salary > 50000;

SELECT * FROM HighSalaryEmps; -- Virtual query
```

Java Memory Cache Example (Using HashMap for Simplicity)

Java

```
import java.util.HashMap;
import java.util.Map;

public class MemoryCache {
    private Map<String, String> cache = new HashMap<>();

    public String get(String key) {
        return cache.get(key); // Fast lookup
    }

    public void put(String key, String value) {
        cache.put(key, value); // Store in memory
    }

    public static void main(String[] args) {
        MemoryCache cache = new MemoryCache();
        cache.put("user1", "John Doe");
        System.out.println(cache.get("user1")); // Output: John Doe
    }
}
```

7. Exception, What is Exception Handling? Its Types

Detailed Explanation

- **Exception**: An event disrupting normal program flow (e.g., divide by zero). In Java, it's an object derived from Throwable (Error or Exception).
- **Exception Handling**: Mechanism to detect, handle, and recover from exceptions using try-catch-finally.
    1. **try**: Block where exception might occur.

2. **catch**: Handles specific exception types.
3. **finally**: Always executes (cleanup).
4. **throw/throws**: Manual throw; declare in method signature.

- **Types**:
  1. **Checked**: Compile-time (e.g., IOException); must handle.
  2. **Unchecked**: Runtime (e.g., NullPointerException); optional.
  3. **Error**: System-level (e.g., OutOfMemoryError); usually unrecoverable.

Java Code Example

Java

```
public class ExceptionHandling {
    public static void divide(int a, int b) throws ArithmeticException { //
throws for checked
        if (b == 0) {
            throw new ArithmeticException("Division by zero!");
        }
        System.out.println(a / b);
    }

    public static void main(String[] args) {
        try {
            divide(10, 0); // Throws unchecked
        } catch (ArithmeticException e) { // Catch unchecked
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Cleanup done.");
        }

        // Checked example
        try {
            // File read code here
        } catch (IOException e) { // Must handle checked
            System.out.println("IO Error: " + e.getMessage());
        }
    }
}
```

- **Output**:

  text

```
Error: Division by zero!
```

- `Cleanup done.`

8. Threads, Its Types

Detailed Explanation

- **Thread**: Lightweight process for concurrent execution. Improves performance (e.g., UI responsiveness).
    1. **Creation**: Extend Thread class or implement Runnable.
    2. **Lifecycle**: New → Runnable → Running → Blocked/Waiting → Terminated.
    3. **Synchronization**: Use synchronized for thread-safety.
- **Types**:
    1. **User Threads**: Created by app; run until completion or daemon=false.
    2. **Daemon Threads**: Background (e.g., GC); stop when user threads end.
    3. **Main Thread**: Entry point.

Java Code Example

Java

```
class MyThread extends Thread {
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println(Thread.currentThread().getName() + ": " + i);
        }
    }
}

public class ThreadsExample {
    public static void main(String[] args) {
        Thread t1 = new MyThread();
        t1.setDaemon(true); // Daemon type
        t1.start();

        Thread t2 = new Thread(() -> { // Lambda for Runnable
            for (int i = 0; i < 3; i++) {
                System.out.println("Runnable Thread: " + i);
            }
        });
        t2.start();
    }
}
```

- **Output** (interleaved):

  text

```
Thread-0: 0
Runnable Thread: 0
```

- `...`
- **How it works**: run() executes concurrently; daemon dies with main.

9. All OOP Concepts

Detailed Explanation

OOP pillars in Java:

1. **Encapsulation**: Bundle data (fields) and methods; hide internals via access modifiers (private, public). Use getters/setters.
2. **Inheritance**: Reuse code via extends. Single inheritance for classes; multiple via interfaces.
3. **Polymorphism**: One interface, multiple forms. Compile-time (overloading), runtime (overriding).
4. **Abstraction**: Hide complexity; show essentials. Via abstract classes/interfaces.

Java Code Example

Java

```java
// Abstraction
abstract class Animal {
    abstract void sound(); // Must implement
}

// Inheritance & Polymorphism
class Dog extends Animal {
    @Override
    void sound() { // Overriding
        System.out.println("Bark");
    }

    void fetch() {} // Specific method
}

// Encapsulation
class Person {
    private String name; // Hidden

    public String getName() { return name; } // Getter
```

```
    public void setName(String name) { this.name = name; } // Setter
}

public class OOPExample {
    public static void main(String[] args) {
        Animal dog = new Dog(); // Polymorphism
        dog.sound(); // Output: Bark

        Person p = new Person();
        p.setName("Amol"); // Encapsulation
        System.out.println(p.getName());
    }
}
```

10. Why Multiple Inheritance is Not Supported? (Detailed with Code)

Detailed Explanation

Multiple inheritance (inheriting from >1 class) is not supported in Java for classes to avoid **diamond problem**: Ambiguity if two parent classes have the same method (which to call?).

- **Solution**: Interfaces allow multiple (no implementation conflict pre-Java 8; default methods added later with resolution rules).
- **Pros of Restriction**: Simplicity, no ambiguity.
- **Cons**: Forces composition over inheritance.
- **C++ Supports It**: Uses virtual inheritance to resolve diamond.

Java Code Example (Diamond Problem Simulation)

Java

```
// If allowed (not in Java):
/*
class A { void show() { System.out.println("A"); } }
class B { void show() { System.out.println("B"); } }
class C extends A, B { } // Error: Which show()?

C obj = new C();
obj.show(); // Ambiguous!
*/

// Java Way: Interfaces
interface Flyable {
    default void move() { System.out.println("Flying"); }
}
```

```java
interface Swimmable {
    default void move() { System.out.println("Swimming"); }
}

class Duck implements Flyable, Swimmable {
    public void move() { // Override to resolve
        System.out.println("Duck moves both ways");
    }
}

public class MultipleInheritance {
    public static void main(String[] args) {
        Duck duck = new Duck();
        duck.move(); // Output: Duck moves both ways
    }
}
```

- **How it works**: Explicit override resolves conflict. Use composition (has-a) for complex hierarchies.

1. Palindrome

A palindrome is a string or sequence that reads the same forward and backward (ignoring spaces, punctuation, and case). In an interview, you'd typically be asked to check if a given string is a palindrome.

Detailed Explanation

- **Approach**: Compare characters from the start and end of the string, moving toward the center. If all pairs match, it's a palindrome. Handle edge cases like empty strings (true) or single characters (true).
- **Time Complexity**: O(n), where n is the string length.
- **Space Complexity**: O(1) for in-place comparison.

Python Code Example

Python

```
import re

def is_palindrome(s):
    if not s:
        return True  # Edge case
    # Clean non-alphanumeric chars and convert to lowercase
    cleaned = re.sub(r'[^a-zA-Z0-9]', '', s.lower())
    left, right = 0, len(cleaned) - 1
```

```
    while left < right:
        if cleaned[left] != cleaned[right]:
            return False
        left += 1
        right -= 1
    return True

# Test
input_str = "A man a plan a canal Panama"
print(is_palindrome(input_str))  # Output: True
```

- **How it works**: The while loop checks pairs from both ends. Cleaning the string handles real-world inputs.

2. Find Unique Characters in a Char Array Using For Loop

Given a char array (list in Python) with duplicates (e.g., arr = ['a', 'b', 'a', 'c', 'b']), find and print only unique characters. Use only for loops (no advanced data structures like sets for simplicity, as per "for loop" constraint).

Detailed Explanation

- **Approach**: Use nested for loops to compare each character with all others. If no match is found, it's unique. Track visited indices to avoid re-checking.
- **Time Complexity**: $O(n^2)$, where n is array length (acceptable for small arrays).
- **Space Complexity**: O(1).

Python Code Example

Python

```
def find_unique_chars(arr):
    n = len(arr)
    visited = [False] * n  # Track if char is processed

    for i in range(n):
        if visited[i]:
            continue  # Skip if already processed

        is_unique = True
        for j in range(i + 1, n):
            if arr[i] == arr[j]:
                is_unique = False
                visited[j] = True  # Mark duplicate as visited
                break
```

```
        if is_unique:
            print(arr[i], end=" ")
        visited[i] = True

# Test
arr = ['a', 'b', 'a', 'c', 'b']
find_unique_chars(arr)  # Output: c
```

- **How it works**: Outer loop picks a char; inner loop checks for duplicates ahead. visited prevents redundant checks.

3. Swapping Program

Swap two variables without using a temporary variable (common twist). Basic version uses temp; advanced uses arithmetic or XOR.

Detailed Explanation

- **Basic Approach (with temp)**: Use a third variable.
- **Without Temp (Arithmetic)**: For integers, a = a + b; b = a - b; a = a - b;.
- **Without Temp (XOR)**: a = a ^ b; b = a ^ b; a = a ^ b; (bitwise, works for ints).
- **Limitations**: Arithmetic can cause overflow; XOR doesn't work for same values.

Python Code Example (Multiple Methods)

Python

```
def swap_with_temp(a, b):
    temp = a
    a = b
    b = temp
    print(f"With temp: a={a}, b={b}")

def swap_arithmetic(a, b):
    a = a + b
    b = a - b
    a = a - b
    print(f"Arithmetic: a={a}, b={b}")

def swap_xor(a, b):
    a = a ^ b
    b = a ^ b
    a = a ^ b
    print(f"XOR: a={a}, b={b}")
```

```
# Test
x, y = 10, 20
swap_with_temp(x, y)
x, y = 10, 20  # Reset
swap_arithmetic(x, y)
x, y = 10, 20  # Reset
swap_xor(x, y)
```

- **Output**:

  text

```
With temp: a=20, b=10
Arithmetic: a=20, b=10
```

- `XOR: a=20, b=10`
- **How it works**: Each method achieves swap; choose based on constraints (e.g., no overflow for arithmetic). Note: Python allows a, b = b, a natively, but we show manual methods.

4. SQL Problems on DELETE, GROUP BY, HAVING (Basic Examples in MySQL)

Assuming basic problems like deleting duplicates, grouping sales by category with conditions. I'll provide 4-5 example queries with explanations. Use a sample table: employees (id INT PRIMARY KEY, name VARCHAR(50), dept VARCHAR(50), salary DECIMAL(10,2)).

Detailed Explanation

- **DELETE**: Removes rows matching conditions. Use WHERE for specifics.
- **GROUP BY**: Aggregates data (e.g., COUNT, SUM) by column.
- **HAVING**: Filters groups (like WHERE but for aggregates).

Example Queries (MySQL Syntax)

1. **Delete employees with salary < 50000**:

   SQL

2.

   ```
   DELETE FROM employees WHERE salary < 50000;
   ```

   - **Explanation**: Basic DELETE with WHERE.
3. **Group by department, count employees, having > 5**:

   SQL

```
SELECT dept, COUNT(*) AS emp_count
FROM employees
GROUP BY dept
```

4. `HAVING COUNT(*) > 5;`
    - **Explanation**: Groups by dept; HAVING filters groups with >5 employees.
5. **Delete duplicates (keep one per name, using MySQL-specific approach)**:

   SQL

```
DELETE e1 FROM employees e1
INNER JOIN employees e2
```

6. `WHERE e1.id > e2.id AND e1.name = e2.name;`
    - **Explanation**: Self-join to identify duplicates; delete higher ID. (Alternative: Use CREATE TEMPORARY TABLE for complex cases.)
7. **Group salaries by dept, sum > 100000**:

   SQL

```
SELECT dept, SUM(salary) AS total_salary
FROM employees
GROUP BY dept
```

8. `HAVING SUM(salary) > 100000;`
    - **Explanation**: Aggregates sum per group; HAVING filters high totals.
9. **Delete from multiple conditions (e.g., low salary in specific dept)**:

   SQL

```
DELETE FROM employees
```

10. `WHERE salary < 40000 AND dept = 'IT';`
    - **Explanation**: Combines WHERE clauses with AND.

5. What is Stored Procedure? What is "IN", "OUT"?

Detailed Explanation

- **Stored Procedure**: A pre-compiled SQL code block stored in the database. It can accept parameters, perform operations (queries, updates), and return results. Benefits: Reusability, performance (compiled once), security (hides logic).
    - Syntax: DELIMITER // CREATE PROCEDURE proc_name(parameters) BEGIN ... END // DELIMITER ;
    - Example Use: CALL proc_name(args);
- **IN Parameter**: Input-only; passes data to the procedure (default mode). Value can't be changed inside.
- **OUT Parameter**: Output-only; returns data from procedure to caller. Procedure sets its value.
- **INOUT**: Both input and output; can read/modify.

MySQL Example

SQL

```
DELIMITER //
CREATE PROCEDURE GetEmployeeSalary(IN emp_id INT, OUT total_salary
DECIMAL(10,2))
BEGIN
    SELECT salary INTO total_salary
    FROM employees
    WHERE id = emp_id;
END //
DELIMITER ;

-- Call (in MySQL client)
SET @sal = 0;
CALL GetEmployeeSalary(101, @sal);
SELECT @sal; -- Outputs the salary
```

- **How it works**: IN emp_id fetches data; OUT total_salary returns it.

Python Integration (Using mysql-connector-python)

Python

```
import mysql.connector

# Assume connection setup
conn = mysql.connector.connect(host='localhost', user='root', password='pass',
database='testdb')
cursor = conn.cursor()

# Call stored procedure
out_salary = None  # Placeholder for OUT
cursor.callproc('GetEmployeeSalary', (101, out_salary))
```

```python
for result in cursor.stored_results():
    out_salary = result.fetchone()[0]
print(out_salary)  # Outputs the salary

cursor.close()
conn.close()
```

6. What is View? Memory Cache

Detailed Explanation

- **View**: A virtual table based on a SQL query. It doesn't store data (unless materialized); simplifies complex queries, enhances security (hide columns). Syntax: CREATE VIEW view_name AS SELECT ...;
    - Pros: Reusability, abstraction.
    - Cons: Can be slower for large data.
- **Memory Cache**: Temporary storage in RAM for frequently accessed data to speed up retrieval (e.g., Redis, but here using dict for simplicity). Reduces database hits.
    - **How it Works**: Key-value store; cache hit = fast read; miss = fetch from source and store.
    - **Types**: In-memory (volatile), distributed.

MySQL View Example

SQL

```sql
CREATE VIEW HighSalaryEmps AS
SELECT name, salary
FROM employees
WHERE salary > 50000;

SELECT * FROM HighSalaryEmps; -- Virtual query
```

Python Memory Cache Example (Simple Dict)

Python

```python
class MemoryCache:
    def __init__(self):
        self.cache = {}

    def get(self, key):
        return self.cache.get(key)  # Fast lookup

    def put(self, key, value):
        self.cache[key] = value  # Store in memory
```

```
# Test
cache = MemoryCache()
cache.put("user1", "John Doe")
print(cache.get("user1"))  # Output: John Doe
```

Python + MySQL Cache Integration

Python

```
import mysql.connector

def get_user_from_cache_or_db(cache, conn, user_id):
    if user_id in cache:
        return cache[user_id]  # Cache hit
    cursor = conn.cursor()
    cursor.execute("SELECT name FROM users WHERE id = %s", (user_id,))
    result = cursor.fetchone()
    if result:
        cache[user_id] = result[0]
    cursor.close()
    return result[0] if result else None

# Usage
cache = {}
conn = mysql.connector.connect(...)  # Setup
print(get_user_from_cache_or_db(cache, conn, 1))
```

7. Exception, What is Exception Handling? Its Types

Detailed Explanation

- **Exception**: An event disrupting normal program flow (e.g., divide by zero). In Python, it's raised as an object (e.g., ValueError).
- **Exception Handling**: Mechanism to detect, handle, and recover from exceptions using try-except-finally-else.
    1. **try**: Block where exception might occur.
    2. **except**: Handles specific exception types.
    3. **else**: Runs if no exception.
    4. **finally**: Always executes (cleanup).
    5. **raise**: Manual raise.
- **Types**:
    1. **Built-in**: Standard (e.g., ValueError, TypeError).
    2. **Custom**: User-defined via subclassing Exception.
    3. **Warning**: Non-fatal (e.g., DeprecationWarning).

Python Code Example

Python

```
class CustomException(Exception):
    pass

def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero!")  # Built-in
    return a / b

try:
    result = divide(10, 0)  # Raises exception
except ValueError as e:  # Handle built-in
    print(f"Error: {e}")
else:
    print(f"Result: {result}")
finally:
    print("Cleanup done.")

# Custom
try:
    raise CustomException("Custom error!")
except CustomException as e:
    print(f"Custom: {e}")
```

- **Output**:

  text

```
Error: Division by zero!
Cleanup done.
```

- `Custom: Custom error!`

8. Threads, Its Types

Detailed Explanation

- **Thread**: Lightweight process for concurrent execution. Improves performance (e.g., I/O-bound tasks).
  1. **Creation**: Use threading.Thread or concurrent.futures.
  2. **Lifecycle**: Start → Run → Join/Stop.

    3. **Synchronization**: Use Lock for thread-safety.
- **Types**:
    1. **User Threads**: Main app threads; run until completion.
    2. **Daemon Threads**: Background (e.g., timers); stop when user threads end.
    3. **Main Thread**: Entry point.

Python Code Example (Using threading)

Python

```
import threading
import time

def thread_func(name, delay):
    for i in range(3):
        print(f"{name}: {i}")
        time.sleep(delay)

# User thread
t1 = threading.Thread(target=thread_func, args=("Thread-1", 1))
t1.start()

# Daemon thread
t2 = threading.Thread(target=thread_func, args=("Daemon-Thread", 0.5))
t2.daemon = True  # Daemon type
t2.start()

t1.join()  # Wait for user thread
print("Main ended.")
```

- **Output** (interleaved):

  text

```
Thread-1: 0
Daemon-Thread: 0
Daemon-Thread: 1
...
```

- `Main ended.`
- **How it works**: target runs concurrently; daemon dies with main.

9. All OOP Concepts

Detailed Explanation

OOP pillars in Python:

1. **Encapsulation**: Bundle data (attributes) and methods; hide internals via _ (convention) or __ (name mangling). Use properties for getters/setters.
2. **Inheritance**: Reuse code via subclassing.
3. **Polymorphism**: One interface, multiple forms. Duck typing; overriding methods.
4. **Abstraction**: Hide complexity; show essentials. Via abstract base classes (abc module).

Python Code Example

Python

```python
from abc import ABC, abstractmethod

# Abstraction
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass  # Must implement

# Inheritance & Polymorphism
class Dog(Animal):
    def sound(self):  # Overriding
        print("Bark")

    def fetch(self):
        pass  # Specific method

# Encapsulation
class Person:
    def __init__(self):
        self._name = None  # Protected (convention)

    @property
    def name(self):  # Getter
        return self._name

    @name.setter
    def name(self, value):  # Setter
        self._name = value

# Test
dog = Dog()  # Polymorphism (Animal reference)
```

```
dog.sound()  # Output: Bark

p = Person()
p.name = "Amol"  # Encapsulation
print(p.name)  # Output: Amol
```

10. Why Multiple Inheritance is Not Supported? (Detailed with Code)

Detailed Explanation

Multiple inheritance (inheriting from >1 class) **is supported in Python** for classes, unlike Java. However, it can lead to the **diamond problem**: Ambiguity if two parent classes have the same method (which to call?).

- **Resolution**: Python uses Method Resolution Order (MRO) via C3 linearization to determine call order (depth-first, left-to-right).
- **Pros**: Flexibility.
- **Cons**: Complexity; prefer composition.
- **When to Avoid**: Deep hierarchies; use mixins for traits.

Python Code Example (Diamond Problem & Resolution)

Python

```
class A:
    def show(self):
        print("A")

class B:
    def show(self):
        print("B")

class C(A, B):  # Multiple inheritance
    pass

class D(B, A):  # Different order
    pass

# MRO check
print(C.__mro__)  # (<class '__main__.C'>, <class '__main__.A'>, <class
'__main__.B'>, <class 'object'>)
print(D.__mro__)  # (<class '__main__.D'>, <class '__main__.B'>, <class
'__main__.A'>, <class 'object'>)

c = C()
```

```
c.show()  # Output: A (A before B in MRO)

# Diamond: If shared grandparent
class Grand:
    def move(self):
        print("Grand move")

class Parent1(Grand):
    def move(self):
        print("Parent1 move")

class Parent2(Grand):
    def move(self):
        print("Parent2 move")

class Child(Parent1, Parent2):
    pass

child = Child()
child.move()  # Output: Parent1 move (MRO: Child -> Parent1 -> Parent2 ->
Grand)
print(Child.__mro__)
```

- **Output**:
  text

```
(<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <class
'object'>)
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.A'>, <class
'object'>)
A
(<class '__main__.Child'>, <class '__main__.Parent1'>, <class
'__main__.Parent2'>, <class '__main__.Grand'>, <class 'object'>)
```

- `Parent1 move`
- **How it works**: MRO resolves conflicts predictably. For interfaces-like, use ABC. Use super() for cooperative inheritance.